CAMPUSCARE

Student Safety & Emergency Response System

A project story — from an emergency-contact manager to an integrated campus safety platform

Project	CampusCare
Implementation	C++ backend with file-based persistence; Qt6 Widgets frontend planned/in progress
Document type	Development journey and technical overview
Version	Storyline draft — November 2026

Prepared as a narrative record of the project’s idea, implementation, technical decisions, challenges, and next stage.

How to read this story

CampusCare did not begin as a finished platform. It grew in stages: first by solving a focused problem, then by connecting separate modules, then by adding algorithms and operational workflows, and finally by preparing for a graphical interface. This document follows that development path rather than presenting the system as if every planned feature had existed from day one.

A note on implementation status

The backend features described as implemented reflect the project progress shared during development. The Qt6 frontend is the next development stage and is not represented here as completed. Some operational details, such as exact deployment configuration and final team contributions, should be filled in after the final build and team review.

Story chapters

- 01 — The problem: campus emergencies need coordination
- 02 — The first version: emergency contacts and persistent storage
- 03 — Bringing incident reporting into the same system
- 04 — Adding the campus map and route intelligence
- 05 — From incidents to response: queues and ambulance dispatch
- 06 — Building the people and communication layer
- 07 — The technical foundation: C++, data structures, algorithms, and files
- 08 — Challenges, decisions, and lessons
- 09 — The next chapter: a Qt6 desktop interface
- 10 — Current state, future scope, and team details

01 — The problem: campus emergencies need coordination

A campus is a small community, but it is also a complex physical environment. Students, faculty, staff, visitors, buildings, laboratories, hostels, gates, and medical facilities are spread across a connected space. When an emergency occurs, the challenge is not only identifying that something has happened. People also need a way to report it, find relevant information, prioritize urgent cases, and coordinate a response.

CampusCare was conceived as a student safety and emergency response system that brings several of these tasks into one place. Instead of treating emergency contacts, incident reports, campus locations, and response operations as unrelated activities, the project aims to connect them through a single software system.

The guiding idea

> *Make it easier to record an emergency, understand its urgency, locate the relevant place, and support a coordinated response — while keeping useful student and communication information accessible to the appropriate users.*

The project is a college software engineering effort, so it also has a learning goal: to apply object-oriented programming, data structures, algorithms, file handling, modular design, and version control to a practical problem.

02 — The first version: emergency contacts and persistent storage

The first working foundation focused on emergency contacts. A user could add a contact, display saved contacts, search for a contact, and delete one. The contact record was represented with fields such as name, phone number, and category.

The project used separate header and source files to keep responsibilities organized. Contact-related operations were handled by a manager class, while file operations were separated into a file manager. This introduced a useful design boundary: one part of the program manages contact behaviour, and another handles reading and writing the stored records.

For persistence, the initial version used plain-text files. Contact records were stored one per line, with fields separated by a pipe character. This made the data easy to inspect and helped the team understand file streams, parsing, and the difference between in-memory data and data saved between runs.

The early development experience also included debugging a file-opening issue. The application could report that it could not open a contacts file, even while some operations appeared to work. Investigating the working directory and file location helped establish an important lesson: a program’s relative file paths depend on where it is launched from.

That first version was intentionally modest. Its value was that it provided a working base that could be expanded rather than a collection of unrelated demonstrations.

03 — Bringing incident reporting into the same system

The next step was to move beyond storing emergency contacts and introduce incident reporting. A separate incident-reporting implementation was brought into the project and integrated with the existing application. This created an important engineering task: combine separate codebases without leaving multiple competing program entry points or breaking the original contact functionality.

An incident record contains information such as an ID, reporter, location, incident type, severity, description, owner, and status. Incident IDs begin at 1001 in the current implementation. Severity is represented in four levels: Low, Medium, High, and Critical.

The incident workflow expanded the system in several directions. Reports could be created, searched, viewed by the person who submitted them, sorted, and updated as their status changed. Hash-based lookup supports finding an incident by its identifier, while counters and reporting functions help summarize pending, resolved, and severity-based activity.

Prioritizing urgent work

Not every incident should wait in the same order. CampusCare uses a normal FIFO queue for regular incidents and a priority queue for high-severity incidents. The priority comparator gives higher severity precedence; when severities are equal, the lower incident ID is handled first. This provides a clear example of how the choice of data structure can reflect a real operational rule.

Integration also exposed a design limitation. Some original manager functions interact directly with the console through `cin` and `cout`. That is convenient for a text menu, but a graphical interface needs functions that accept values as parameters and return results. The later GUI architecture therefore needs an adapter layer and carefully designed non-interactive interfaces; it should not simply try to make a Qt form behave like console input.

04 — Adding the campus map and route intelligence

A report is more useful when the system can reason about where it happened. CampusCare therefore gained a campus graph that models locations as vertices and walkable connections as weighted edges. Each edge represents a path and its distance in metres. The paths are treated as bidirectional in the current campus model.

The graph was populated from a campus sketch supplied during development. Locations include New Audi, Fashion Department, CSE Department, GEHU Ground, Open Audi, Basement, Cafeteria, Gate 1, Tuck Shop, Mechanical Lab, Main Gate, Girls Hostel, and Civil Lab. The model also includes medical destinations: Tyagi Clinic and Hospital. Hospital is registered as a hospital destination in the graph.

The graph module supports several useful operations:

- **Display campus:** list known locations and identify hospital locations.
- **BFS:** explore reachable locations level by level.
- **DFS:** traverse the graph depth-first.
- **Dijkstra’s shortest path:** find a minimum-distance route through weighted paths.
- **Nearest-hospital route:** evaluate routes to registered hospital destinations and return a route and distance.

For example, the shortest route from Main Gate to Tyagi Clinic in the supplied map is 750 metres: Main Gate to Gate 1 (350 m), then Gate 1 to Tyagi Clinic (400 m). This is a route calculation over the entered graph data, not a live navigation service.

This distinction matters. The graph is only as accurate as the locations and distances entered into it. It currently represents the project’s mapped campus paths; it does not automatically read GPS data, indoor positioning, live traffic, or changing access restrictions.

05 — From incidents to response: queues and ambulance dispatch

Once incidents could be recorded and locations could be connected, the next question was how to represent a response. CampusCare includes an emergency system that separates regular incidents from high-priority incidents using the two queue types described earlier.

The project also includes an ambulance dispatch module. Ambulances are represented through a vehicle abstraction and an Ambulance type derived from the Vehicle base class. Fleet information is stored in `ambulances.txt`, while dispatch activity is recorded in `dispatch_log.txt`. Dispatch and completion operations report success through the returned incident identifier; a failure is represented by -1.

This module gives the project a model for assigning and tracking emergency vehicles. It should be understood as a software workflow and simulation based on the project's stored fleet and dispatch records. It is not connected to a real ambulance provider or an external emergency-dispatch network.

06 — Building the people and communication layer

As the system grew, it became clear that incidents are not the only information a campus safety application may need to manage. The project expanded to include student profiles, account management, concerns, messages, notifications, and an administrative overview.

Student profiles

A student profile stores a student ID, name, department, phone number, blood group, gender, emergency contacts, and account status. Student records are stored in `students.txt`. The current profile structure uses a vector for emergency contacts and a Boolean active/inactive flag.

User roles and login

The current account flow distinguishes students, faculty, and administrators. Students and faculty use an ID-only login flow, while administrators use an ID and the existing administrator password. Account activation status is checked for student and faculty accounts. New users can be registered, and the project includes operations to deactivate and reactivate accounts.

The existing login rules are part of the project's behaviour and should be preserved when the GUI is introduced. The frontend should present the right fields for each role, but the underlying authentication and authorization rules must remain controlled by the backend.

Concerns, messaging, and notifications

Concern management provides a place for campus-related concerns. Messaging and notifications add communication-oriented modules, while role-based menu permissions determine which operations are available to different users. An administrative dashboard brings selected system information together for oversight.

These modules move CampusCare closer to a connected campus application: not merely a form for recording emergencies, but a platform with user accounts, operational records, and communication features.

07 — The technical foundation: C++, data structures, algorithms, and files

CampusCare's backend is implemented in C++. Its design uses multiple header and source files so that data structures, manager classes, and application flow can be maintained separately. The project uses object-oriented programming concepts such as classes, encapsulation, and inheritance, alongside the C++ Standard Library.

Core data structures and where they fit

Structure / technique	Use in CampusCare
Structs and classes	Represent contacts, incidents, students, messages, notifications, concerns, vehicles, and managers.
Vector	Store collections such as contacts, profile emergency contacts, and graph adjacency lists.
Queue	Process regular incidents in first-in, first-out order.
Priority queue	Give higher-severity incidents precedence over lower-severity ones.
Hash-based lookup	Find incident records efficiently by ID.
Map / adjacency list	Represent campus locations and weighted connections.
BFS and DFS	Traverse the campus graph using breadth-first and depth-first strategies.
Dijkstra's algorithm	Calculate shortest paths in the weighted campus graph.
Sorting	Order incident reports for review.
File streams	Persist records in text files between program runs.

File-based persistence

The backend uses pipe-delimited text files for persistence. Examples include contacts, incidents, students, users, ambulance fleet information, and dispatch logs, with additional files supporting the other modules. This keeps the project self-contained and makes stored records easy to inspect during development.

Text files are a deliberate project-stage choice, not a claim that they are ideal for every production deployment. As the system grows, a database could provide stronger validation, concurrent access, transactions, and more robust querying. The current scope retains text-file storage.

Modular design

The project separates features into manager and system modules — for example, ContactManager, FileManager, IncidentManager, EmergencySystem, CampusGraph, AmbulanceDispatch, StudentManager, ConcernManager, MessageManager, and NotificationManager. The goal is to keep responsibilities understandable and make later changes less likely to affect unrelated modules.

A single console application entry point currently coordinates the backend menu. The planned GUI will have its own entry point inside a separate `gui/` directory so the console application can remain independently buildable.

08 — Challenges, decisions, and lessons

The development journey involved more than writing feature code. Several of the most useful lessons came from integration, debugging, and project organization.

Combining separate modules

Incident reporting began as a separate implementation. Bringing it into CampusCare required consolidating the application flow and managing source files, headers, and build commands. One practical lesson was to compile from the directory containing the source files or provide the correct paths; running a build command from the wrong folder led to “No such file or directory” errors.

File paths and persistence

Text-file operations depend on the program’s working directory. A file may exist in the project but still fail to open if the application is launched from a different directory. Testing add, display, search, and reload behaviour helped validate that records were actually being saved and loaded.

Git and GitHub

Version control was a learning challenge, particularly initializing a repository, understanding untracked and staged files, configuring a remote, working with branches, and pushing changes. The project used a dedicated testing branch rather than treating every development change as a direct update to the main branch. A `.gitignore` file was also used to keep local test executables and password experiments out of the repository.

Working with a growing codebase

As more modules were added, it became increasingly important to understand file dependencies, method signatures, build commands, and the effect of changes across the project. The project’s next stage makes this especially important: a graphical interface should not destabilize a working backend.

Security and scope decisions

A password-hashing experiment was explored separately, but it was not integrated into the current CampusCare implementation. The current administrator password handling should therefore not be described as securely hashed. This is a known limitation and a possible future improvement if the project’s scope and requirements change.

What these challenges taught

- Debugging requires checking the environment and working directory, not just the source code.
- Integration is a design task: compatible modules need a shared application flow and consistent interfaces.
- Version control is part of software development, not an optional final step.
- A GUI should reuse backend rules and data rather than create a second, conflicting implementation.

09 — The next chapter: a Qt6 desktop interface

With the backend substantially developed, the next stage is a desktop interface built using Qt6 Widgets and C++. The decision is to keep the project within the C++ ecosystem rather than introduce a separate Python or web frontend. The existing text-file storage approach is also being retained for this stage.

The intended visual direction is a modern, professional application with both light and dark themes. The login screen is planned around a university-style composition: institutional branding, a campus-building background, and a centered login card. The actual campus photograph can replace a temporary placeholder when it is available.

The current role-based login behaviour is to remain unchanged: student ID only, faculty ID only, and administrator ID plus the existing password. The interface should reveal the password field for the administrator role and should not add a CAPTCHA or introduce new student/faculty passwords.

Proposed GUI architecture

- **BackendBridge:** an adapter layer that exposes GUI-friendly methods, accepting form values and returning structured results. It should reuse backend business rules and persistence rather than duplicate them.
- **SessionManager:** a focused place to represent the signed-in user and role, while preserving existing authentication rules.
- **ThemeManager:** a central place to apply light and dark Qt stylesheets.
- **Page-based interface:** separate screens for the dashboard and individual features, with navigation controlled by the user's role and permissions.

A key technical issue is that some current backend methods are console-oriented and read from `cin` or write to `cout`. Qt forms cannot safely substitute for those interactions. The bridge will need clean, non-interactive interfaces, introduced incrementally and reviewed before any backend changes are made.

The first GUI milestone is deliberately small: create a separate Qt application inside `gui/`, launch a basic main window, establish light and dark themes, and confirm that the original console application still builds. Login and backend integration should follow only after this foundation is stable.

The principle for this phase

> *Build alongside the backend, not over it. Keep the console application working, change one layer at a time, and verify each milestone before moving on.*

10 — Current state, future scope, and team details

Current backend scope

Area	Current project capability
Emergency contacts	Add, display, search, delete, and persist contact records.
Incident management	Create and manage incident reports, search by ID, view personal reports, sort, update status, and count report states/severities.
Emergency handling	FIFO queue for regular incidents and severity-based priority queue for urgent incidents.
Campus graph	Campus locations, weighted paths, BFS, DFS, shortest routes, and nearest registered hospital routing.
Ambulance dispatch	Fleet records, dispatch workflow, completion workflow, and dispatch log.
Profiles and accounts	Student profiles, role-aware login, account status, registration, deactivation, and reactivation.
Communication	Concern management, messaging, notifications, and role-based menu permissions.
Administration	Administrative dashboard and system summaries.

Planned / future improvements

- Complete the Qt6 GUI foundation, then implement the role-aware login screen.
- Build role-specific dashboards and feature pages, connecting them through the BackendBridge.
- Refactor console-bound operations into reusable non-interactive methods where necessary, with review and tests for each change.
- Replace the temporary login background with an approved campus photograph and add the final institutional branding.
- Validate campus path distances and location names against the actual campus layout.
- Improve input validation, error handling, and automated testing across modules.
- Consider stronger credential storage and database-backed persistence as future scope, if permitted.

Known limitations to state honestly

- Campus routing uses manually entered graph locations and distances; it is not live GPS navigation.
- Ambulance dispatch is an internal software workflow, not a connection to a real emergency service.
- Data is stored in text files rather than a database.
- Administrator password hashing has not been integrated.
- The Qt6 frontend is a planned/in-progress stage, not a completed feature.

Team members

Add the final team member names, roles, and photographs here after confirming contributions. This section is intentionally left blank for the team to complete.

Team member	Responsibility / contribution
1. _____	_____
2. _____	_____
3. _____	_____
4. _____	_____

Closing note

CampusCare’s story is one of gradual integration. It began with a practical contact-management feature, grew into incident reporting and emergency prioritization, gained a campus graph and dispatch workflow, and expanded to include profiles and communication. The Qt6 interface is the next step in making these capabilities accessible through a unified desktop experience. The project’s strongest outcome is not only the number of modules it contains, but the understanding gained by designing, connecting, testing, and maintaining them as one system.

End of project story